

CURRENT SYSTEM DOCUMENTATION

KPTC Scheduler

origin・tamanegi Ubuntu 24.04 構築・移行手順書

ビルド済み内部ファイルをoriginへ、外部カレンダーをtamanegiへ配置する、初めての担当者向け実作業ガイド

origin 内部スケジューラー	tamanegi 外部カレンダー	1方向 originからtamanegiへ
--	--	---

この資料の基準
文書版 v5.0 / 更新日 2026-09-02 / 実装基準コミット 3f7f37a / 対象OS Ubuntu 24.04

この手順書の読み方

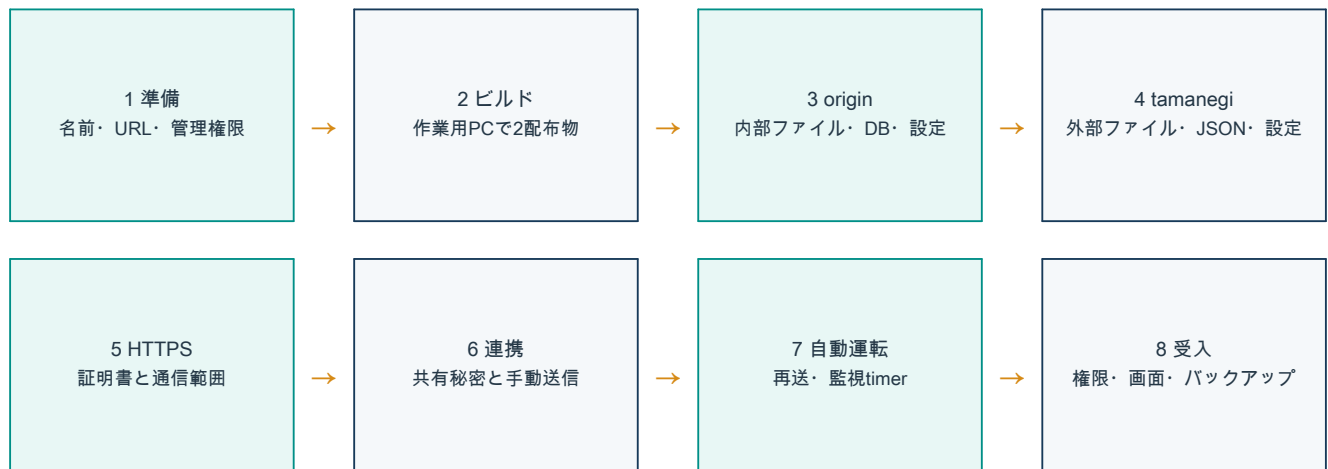
この手順書は、サーバー管理の専門家でない方でも、どの機械で何を行うか、成功したとき何が見えるかを順番に確認できるようにしています。灰色の枠内を1行ずつ実行し、必ず「正常なら」を確認してから次へ進んでください。

環境名	役割	OS	関連ビルド済みファイル
origin	社内専用スケジューラー、SQLite、JSON送信	Ubuntu Server 24.04 LTS	dist-internal 14件
tamanegi	外部公開カレンダー、JSON受信・表示	Ubuntu Server 24.04 LTS	dist-public 13件
作業用PC	ビルド、検査、2台へのアップロード	Windows / macOS / Linux	GitソースとNode.js

大切な約束
originのDBや個人情報をtamanegiへコピーしません。tamanegiには公開可能な3か月JSONだけを置きます。

02

作業の全体順序

**途中で止める条件**

SSH切断、sudo不可、空き容量2GB未満、既存Webへの影響が不明、正式バックアップなし、入力値の意味が不明の場合は先へ進みません。

03

開始前チェックと記入シート

項目	記入欄	例・表記
origin SSH	_____	admin@origin
tamanegi SSH	_____	admin@tamanegi
origin正式URL	_____	社内DNS名。外部公開しない
tamanegi正式URL	_____	外部DNS名。HTTPS公開
社内接続範囲	_____	社内LAN/VPNのIP範囲
初期管理者member_id	_____	既存DBに合わせる
リリース名	20260902-3f7f37a	フォルダー名
バックアップ先	_____	サーバー外の保管先

秘密は記入しない
共有秘密はPDF、作業メモ、メール、GitHubへ書かず、組織の秘密保管場所へ保存します。

04

作業用PCでソースをそろえる

実行場所: 作業用PC

```
cd ~/Documents
git clone https://github.com/Apfelengineer/260725_GW_WEB.git
cd 260725_GW_WEB
git checkout main
git pull --ff-only
git log -1 --oneline
```

正常なら

git logの先頭が3f7f37a、またはそれより新しい承認済みmainです。既存フォルダーを無理に削除せず、新しい空フォルダーを使います。

server-runtime-snapshotはさくら配備済み完成物の照合用です。Ubuntuへの新規配布は、この後に生成するdist-internal / dist-publicを基準にします。

05

作業用PCでビルドする

実行場所: 作業用PC / Node.js 22.13以上

```
node --version
corepack enable
corepack prepare pnpm@latest --activate
pnpm --version
pnpm install --frozen-lockfile
pnpm run check
pnpm test
pnpm run build
```

正常なら

TypeScriptエラー0、テスト8件pass、dist-internal 14件、dist-public 13件が生成されます。

サーバーではビルドしない

originとtamanegiにNode.js・pnpm・ソース・node_modulesは不要です。置くのはビルド後ファイルです。

ビルド後ファイルを確認する

実行場所: 作業用PC

```
find dist-internal -type f | sort
find dist-public -type f | sort
find dist-internal -type f | wc -l
find dist-public -type f | wc -l
```

ディレクトリ	確認内容
dist-internal	index、main asset、内部API・認証・送信CLI・room-config。14件
dist-public	index/reservations、asset、受信・公開PHP、画像。13件
混入禁止	dist-publicにapi.php、auth.php、SQLite、秘密設定がない
組合せ	HTMLが参照するhash付きJS/CSSが同じフォルダーにある

07

配布ファイルの照合値を作る

実行場所: 作業用PC

アップロード中に壊れていないか確かめるため、内容から照合値を作ります。macOS例です。Linuxではshasumをsha256sumへ読み替えます。

```
cd dist-internal
find . -type f -print | sort | xargs shasum -a 256 > ../origin-SHA256SUMS
cd ../dist-public
find . -type f -print | sort | xargs shasum -a 256 > ../tamanegi-SHA256SUMS
cd ..
```

保管

2つのSHA256SUMSを配布物と一緒に保管します。秘密設定は含めません。

08

両サーバーへ基本ソフトを入れる

実行場所: originとtamanegi

```
sudo apt update
sudo apt install -y nginx php8.3-fpm php8.3-cli php8.3-sqlite3 php8.3-curl php8.3-mbstring sqlite3 rsync curl
sudo systemctl enable --now nginx php8.3-fpm
systemctl is-active nginx
systemctl is-active php8.3-fpm
php -v
php -m | grep -E 'curl|mbstring|pdo_sqlite'
```

正常なら

systemctlが2つともactiveで、PHP 8.3とcurl・mbstring・pdo_sqliteが表示されます。

09

originへ内部ファイルを送る

実行場所: 作業用PC

```
rsync -av --delete dist-internal/ ADMIN@origin:/tmp/kptc-origin/  
scp origin-SHA256SUMS ADMIN@origin:/tmp/kptc-origin-SHA256SUMS
```

置き換える部分

ADMINはSSH管理者名へ置き換えます。originという名前で接続できない場合は、管理者から受け取った正式名またはIPアドレスを使います。

10

originでリリースを配置する

実行場所: origin

```
RELEASE=20260902-3f7f37a
sudo mkdir -p /opt/kptc-scheduler/internal/releases/$RELEASE
sudo mkdir -p /var/lib/kptc-scheduler/etc/kptc-scheduler
sudo rsync -a /tmp/kptc-origin/ /opt/kptc-scheduler/internal/releases/$RELEASE/
sudo chown -R root:www-data /opt/kptc-scheduler/internal/releases/$RELEASE
sudo find /opt/kptc-scheduler/internal/releases/$RELEASE -type d -exec chmod 750 {} \;
sudo find /opt/kptc-scheduler/internal/releases/$RELEASE -type f -exec chmod 640 {} \;
sudo ln -sf /opt/kptc-scheduler/internal/releases/$RELEASE /opt/kptc-scheduler/internal/current
```

正常なら

ls -l /opt/kptc-scheduler/internal/current が今回のRELEASEを示します。旧releaseは戻すために残します。

11

originのデータ領域を準備する

実行場所: origin

```
sudo chown -R www-data:www-data /var/lib/kptc-scheduler
sudo chmod 750 /var/lib/kptc-scheduler
sudo chmod 750 /etc/kptc-scheduler
sudo -u www-data test -w /var/lib/kptc-scheduler
echo $?
```

正常なら

最後が0です。実行ファイルはroot管理、予定DBだけwww-dataが書けるように分けます。

既存DB移行

第17章を先に読み、旧環境の書き込み停止とバックアップを済ませます。新規なら第12章へ進みます。

originのWeb用設定を作る

実行場所: origin

sudoedit /etc/kptc-scheduler/internal-env.php を実行し、TAMANEGI_FQDNと秘密値を実環境へ置き換えます。

```
<?php
putenv('KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite');
putenv('KPTC_PUBLIC_AVAILABILITY_MODE=https');
putenv('KPTC_PUBLIC_AVAILABILITY_ENDPOINT=https://TAMANEGI_FQDN/receive-availability.php');
putenv('KPTC_PUBLIC_AVAILABILITY_PAGE_URL=https://TAMANEGI_FQDN/');
putenv('KPTC_PUBLIC_AVAILABILITY_SECRET=REPLACE_WITH_SHARED_SECRET');
putenv('KPTC_PUBLIC_AVAILABILITY_TIMEOUT=10');
putenv('KPTC_SESSION_COOKIE_SECURE=1');
```

```
sudo chown root:www-data /etc/kptc-scheduler/internal-env.php
sudo chmod 640 /etc/kptc-scheduler/internal-env.php
```

注意

共有秘密は第21章で生成し、originとtamanegiへ同じ値を設定します。

13

originのPHP-FPM専用設定

実行場所: origin

sudoedit /etc/php/8.3/fpm/pool.d/kptc-origin.conf で次を作ります。

```
[kptc-origin]
user = www-data
group = www-data
listen = /run/php/php8.3-fpm-kptc-origin.sock
listen.owner = www-data
listen.group = www-data
pm = ondemand
pm.max_children = 8
pm.process_idle_timeout = 10s
clear_env = no
env[KPTC_INTERNAL_CONFIG_FILE] = /etc/kptc-scheduler/internal-env.php
```

```
sudo php-fpm8.3 -t
sudo systemctl restart php8.3-fpm
ls -l /run/php/php8.3-fpm-kptc-origin.sock
```

正常なら
設定検査がsuccessfulで、専用socketが表示されます。

originのNginx設定

実行場所: origin

sudoedit /etc/nginx/sites-available/kptc-origin を実行し、ORIGIN_FQDNを正式名へ置き換えます。

```
server {
    listen 80;
    server_name ORIGIN_FQDN;
    root /opt/kptc-scheduler/internal/current;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
    location = /api.php {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm-kptc-origin.sock;
    }
    location ~ \.php$ { return 404; }
}
```

```
sudo ln -sf /etc/nginx/sites-available/kptc-origin /etc/nginx/sites-enabled/kptc-origin
sudo nginx -t
sudo systemctl reload nginx
curl -I http://ORIGIN_FQDN/
```

正常なら

nginx -tがsuccessful、画面はHTTP 200です。PHPソースが表示されたら直ちに公開を止めます。

15

tamanegiへ外部ファイルを送る

実行場所: 作業用PC

将来追加する室画像を消さないよう、コードと画像を2回に分けて送ります。

```
rsync -av --delete --exclude='*.png' dist-public/ ADMIN@tamanegi:/tmp/kptc-tamanegi/  
rsync -av dist-public/*.png ADMIN@tamanegi:/tmp/kptc-tamanegi/  
scp tamanegi-SHA256SUMS ADMIN@tamanegi:/tmp/kptc-tamanegi-SHA256SUMS
```

理由

最初の行は古いコードを整理しつつPNGを削除対象から外します。2行目でビルドに含まれる最新PNGを上書きします。サーバーだけに追加した室画像は残ります。

tamanegiでリリースを配置する

実行場所: tamanegi

```
RELEASE=20260902-3f7f37a
sudo mkdir -p /opt/kptc-scheduler/public/releases/$RELEASE
sudo mkdir -p /var/lib/kptc-availability/etc/kptc-availability
sudo rsync -a /tmp/kptc-tamanegi/ /opt/kptc-scheduler/public/releases/$RELEASE/
# 既存の差替画像・追加室画像を新releaseへ引継ぐ
sudo cp -f /opt/kptc-scheduler/public/current/*.png /opt/kptc-scheduler/public/releases/$RELEASE/ 2>/dev/null || true
sudo chown -R root:www-data /opt/kptc-scheduler/public/releases/$RELEASE
sudo find /opt/kptc-scheduler/public/releases/$RELEASE -type d -exec chmod 750 {} \;
sudo find /opt/kptc-scheduler/public/releases/$RELEASE -type f -exec chmod 640 {} \;
sudo ln -sfn /opt/kptc-scheduler/public/releases/$RELEASE /opt/kptc-scheduler/public/current
```

追加室画像

旧releaseだけにある追加室画像は新releaseへコピーしてからcurrentを切り替えます。m6/m7/m8はビルドに含まれます。

17

既存SQLiteをoriginへ移す

実行場所: 旧サーバーとorigin / 新規なら第18章へ

```
# 旧サーバー: 利用者の書き込みを止めてから
sqlite3 group-watcher.sqlite 'PRAGMA integrity_check;'
cp -p group-watcher.sqlite group-watcher.sqlite.pre-migration
scp group-watcher.sqlite ADMIN@origin:/tmp/group-watcher.sqlite
```

```
# origin
sudo systemctl stop php8.3-fpm
sudo cp /tmp/group-watcher.sqlite /var/lib/kptc-scheduler/group-watcher.sqlite
sudo chown www-data:www-data /var/lib/kptc-scheduler/group-watcher.sqlite
sudo chmod 640 /var/lib/kptc-scheduler/group-watcher.sqlite
sudo -u www-data sqlite3 /var/lib/kptc-scheduler/group-watcher.sqlite 'PRAGMA integrity_check;'
sudo systemctl start php8.3-fpm
```

正常なら

2回のintegrity_checkがokです。移行前コピーは別媒体にも保存します。

18

originの初期DB・管理者を確認

実行場所: origin

```
curl 'http://ORIGIN_FQDN/api.php?action=bootstrap'  
sudo -u www-data env KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite php  
/opt/kptc-scheduler/internal/current/manage-auth-user-cli.php list
```

管理者がない場合の例です。adminとm1は実値へ置き換えます。

```
sudo -u www-data env KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite php  
/opt/kptc-scheduler/internal/current/manage-auth-user-cli.php create admin m1 admin
```

正常なら

listにrole=admin、正しいmember_id、有効状態が表示されます。パスワードは使いません。

tamanegiの設定とデータ領域

実行場所: tamanegi

```
sudo chown -R www-data:www-data /var/lib/kptc-availability
sudo chmod 750 /var/lib/kptc-availability
sudo chmod 750 /etc/kptc-availability
```

sudoedit /etc/kptc-availability/public-env.php で次を作ります。

```
<?php
putenv('KPTC_PUBLIC_AVAILABILITY_JSON=/var/lib/kptc-availability/public-availability.json');
putenv('KPTC_PUBLIC_AVAILABILITY_SECRET=REPLACE_WITH_SHARED_SECRET');
putenv('KPTC_PUBLIC_AVAILABILITY_STALE_SECONDS=1800');
```

```
sudo chown root:www-data /etc/kptc-availability/public-env.php
sudo chmod 640 /etc/kptc-availability/public-env.php
```

JSONはまだなくてよい

最初の連携前はpublic-availability.jsonがありません。送信成功時にwww-dataが作成します。

20

tamanegiのPHP-FPM・Nginx

実行場所: tamanegi

PHP-FPM専用poolの要点です。

```
[kptc-tamanegi]
user = www-data
group = www-data
listen = /run/php/php8.3-fpm-kptc-tamanegi.sock
listen.owner = www-data
listen.group = www-data
pm = ondemand
pm.max_children = 8
clear_env = no
env[KPTC_PUBLIC_CONFIG_FILE] = /etc/kptc-availability/public-env.php
```

Nginxでは公開PHPだけを許可します。sudoedit /etc/nginx/sites-available/kptc-tamanegi で次を作ります。

```
server {
    listen 80;
    server_name TAMANEGI_FQDN;
    root /opt/kptc-scheduler/public/current;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
    location ~ ^/(receive-availability|public-availability|health-availability)\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm-kptc-tamanegi.sock;
    }
    location ~ \.php$ { return 404; }
}
```

確認

php-fpm8.3 -t、nginx -t、再起動・再読込後、トップ200、health初回503なら正常です。

21

共有秘密を設定し手動送信

実行場所: originとtamanegi

originで次の値を1回だけ生成し、組織の秘密保管場所へ保存します。

```
openssl rand -hex 32
```

- originのinternal-env.phpへ設定します。
- tamanegiのpublic-env.phpへ同じ値を設定します。
- 第22章のsystemd用envにも同じ値を設定します。
- 両サーバーでPHP-FPMを再起動します。

```
# origin
sudo -u www-data env KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php php
/opt/kptc-scheduler/internal/current/publish-availability-cli.php
# 任意のPC
curl -i https://TAMANEGI_FQDN/health-availability.php
```

正常なら
送信CLIがok:true・終了コード0、tamanegi healthがHTTP 200です。

22

originの自動再送設定ファイル

実行場所: originと作業用PC

originで `sudoedit /etc/kptc-scheduler/internal-server.env` を実行し、Web用と同じ値をKEY=VALUE形式で記載します。

```
KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite
KPTC_PUBLIC_AVAILABILITY_MODE=https
KPTC_PUBLIC_AVAILABILITY_ENDPOINT=https://TAMANEGI_FQDN/receive-availability.php
KPTC_PUBLIC_AVAILABILITY_PAGE_URL=https://TAMANEGI_FQDN/
KPTC_PUBLIC_AVAILABILITY_SECRET=REPLACE_WITH_SHARED_SECRET
KPTC_PUBLIC_AVAILABILITY_TIMEOUT=10
KPTC_PUBLIC_AVAILABILITY_STALE_SECONDS=1800
```

```
sudo chown root:root /etc/kptc-scheduler/internal-server.env
sudo chmod 600 /etc/kptc-scheduler/internal-server.env
```

作業用PCで次を実行します。

```
scp deploy/kptc-availability-* ADMIN@origin:/tmp/
```

23

systemd timerを開始する

実行場所: origin

```
sudo cp /tmp/kptc-availability-publish.service /etc/systemd/system/  
sudo cp /tmp/kptc-availability-publish.timer /etc/systemd/system/  
sudo cp /tmp/kptc-availability-monitor.service /etc/systemd/system/  
sudo cp /tmp/kptc-availability-monitor.timer /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now kptc-availability-publish.timer  
sudo systemctl enable --now kptc-availability-monitor.timer  
systemctl list-timers | grep kptc
```

正常なら

publishは約5分ごと、monitorは約10分ごとの次回時刻が表示されます。

```
journalctl -u kptc-availability-publish.service -n 50 --no-pager  
journalctl -u kptc-availability-monitor.service -n 50 --no-pager
```


DNS・HTTPS・通信範囲

実行場所: ネットワーク管理者と両サーバー

方向	アクセス範囲
職員PC -> origin	社内LAN/VPNからHTTPS 443だけ
社外 -> origin	遮断。DNS非公開または接続拒否
origin -> tamanegi	HTTPS 443。JSON POST
一般 -> tamanegi	HTTPS 443。公開カレンダー
SSH 22	管理端末・管理ネットワークだけ

- originは組織の内部証明書または社内認証基盤を使います。
- tamanegiは組織指定証明書、許可時はLet's Encryptを使えます。
- HTTPはHTTPSへ転送し、証明書警告がないことを確認します。
- 両サーバーの時刻差を5分未満にします。

```
# 両サーバーの時刻同期
sudo timedatectl set-ntp true
timedatectl status
# tamanegiでLet's Encryptを使える場合の例
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d TAMANEGI_FQDN
sudo certbot renew --dry-run
```

originを社内認証の内側へ置く

実行場所: 社内システムとorigin

最重要

KPTC Schedulerはパスワードを確認せず、選択ユーザーの権限を付与します。originを誰でも直接開ける状態では本番データを入れません。

- 社内ポータル代理入口配下へ置き、未認証ならログインへ戻します。
- 組織のOIDC/SAML入口で認証済み要求だけをoriginへ渡します。
- 社内LAN/VPNだけに443を許可し、社外経路を遮断します。
- iframeでもorigin直接URLを未認証では開けなくします。

状態	対応動作
未ログイン	originを開けない / 上位ログインへ移動
ログイン済み	ユーザー選択を表示
社外	origin拒否、tamanegi表示可

新しい試験室を外部公開へ追加

実行場所: 管理画面・作業用PC・origin・tamanegi

- 管理画面で所属「試験室」のユーザーを追加し、IDを控えます。
- public/availability-room-config.phpのkptc_public_room_ids()へIDを表示順で追記します。
- 公開名や説明を変える場合はoverridesへ追記します。
- pnpm run build:internal後、dist-internalをoriginへ再配布します。
- <ユーザーID>.pngをtamanegiのcurrent直下へ置き、root:www-data、640にします。
- originでpublish CLIを実行し、公開画面の切替が増えたことを確認します。

作り直し不要
公開画面はrooms配列から自動表示するため、室数追加のための外部画面再実装は不要です。

公開前承認
許可リスト追加前に、室名・画像・問い合わせ先・公開可否を担当者が承認します。

27

更新・元に戻す・バックアップ

実行場所: originとtamanegi

- 更新は新しいreleases/<版>へ置き、検査後にcurrentを切替えます。
- 問題時はcurrentを直前releaseへ戻します。DBは自動では戻りません。
- originのSQLiteと/etc/kptc-schedulerを毎日別媒体へ保存します。
- tamanegiの/etc/kptc-availabilityと追加室画像も保存します。公開JSONは再生成できます。
- 決めた間隔で別の場所へ復元できるか実際に試します。

```
sudo -u www-data sqlite3 /var/lib/kptc-scheduler/group-watcher.sqlite ".backup '/var/backups/kptc/group-watcher-YYYYMMDD.sqlite'"
sqlite3 /var/backups/kptc/group-watcher-YYYYMMDD.sqlite 'PRAGMA integrity_check;'
```

古い版

新しい版とバックアップを確認後、保存期間に従って削除します。currentが指す版は削除しません。

最終受入チェック

実行場所: 職員PC・社外回線・両サーバー

項目	確認内容	結果
origin	HTTPS、社内認証内、ユーザー選択・ゲスト	☐
権限	管理者・一般・試験室・ゲストが仕様どおり	☐
予定	登録・編集・複数操作・D&D確認	☐
tamanegi	許可室、3か月、土日祝、記号・濃灰	☐
JSON	schema v2、rooms、個人情報なし、health 200	☐
再送	2つのtimer active、継続エラーなし	☐
境界	社外からorigin不可、tamanegi可	☐
バックアップ	DB・設定・画像を別媒体へ保存・復元	☐
台帳	URL、担当者、停止・復旧手順	☐

本番開始
全項目が合格し、originの未認証直接アクセスが遮断された時点で開始します。

困ったときの確認順

実行場所: 該当サーバー

症状	原因と対応
画面が開かない	DNS名、curl -l、nginx active、nginx -t
PHPが動かない	php8.3-fpm active、socket、fastcgi_pass
予定保存不可	DB所有者、/var/lib書込、API応答、PHPログ
外部が準備中	origin手動送信、共有秘密、JSON権限
health 503	updatedAt、timer、時刻、30分超停止
署名エラー	秘密値、時刻、HTTPS受信URL
追加室なし	室ID、許可リスト、再ビルド、手動送信
画像なし	<ID>.pngの名前・大文字小文字・権限

```
systemctl status nginx php8.3-fpm
sudo nginx -t
journalctl -u nginx -n 50 --no-pager
journalctl -u php8.3-fpm -n 50 --no-pager
journalctl -u kptc-availability-publish.service -n 50 --no-pager
```

30-1

専門用語をやさしい言葉で説明

最終章 1/4。日常の言葉に置き換えます。

言葉	やさしい説明
サーバー	画面やデータを置き、みんなへ渡すコンピューターです。
内部 / 外部	内部は組織の中だけ、外部はインターネットから見える範囲です。
OS / Ubuntu	OSは機械全体を動かす基本ソフトです。Ubuntu 24.04はLinuxの一種です。
ブラウザー	Webページを見るSafari、Chrome、Edgeなどです。
URL	Webページの場所を表す文字の宛先です。
IPアドレス	ネットワーク上の機械を表す数字の住所です。
ドメイン / FQDN	数字の住所を読みやすい名前にしたものです。FQDNは省略しない正式名です。
DNS	名前からIPアドレスを調べる住所録です。
ポート	用途別の入口番号です。HTTPSは主に443、SSHは22です。
ファイアウォール	通してよい通信だけを通す門です。

専門用語をやさしい言葉で説明

最終章 2/4。日常の言葉に置き換えます。

言葉	やさしい言葉
HTTPS / TLS	Web通信を暗号で守り、相手が正しいか確かめます。
証明書	HTTPSで使う電子的な名札です。
SSH	離れたサーバーへ安全に入り、文字で操作する仕組みです。
ターミナル / コマンド	文字で命令する画面と、そこで入力する短い命令です。
sudo	管理者の力が必要な操作を一時的に行う命令です。
フォルダー / ディレクトリ	ファイルをまとめる入れ物です。
所有者 / 権限	誰が読める・書けるかを決める決まりです。
Nginx	ブラウザの要求を受けるWebサーバーソフトです。
PHP / PHP-FPM	サーバー側処理の言葉と、その処理を実行する仕組みです。
SQLite / DB	決まりに沿って情報を保存する箱です。SQLiteは1ファイルで管理します。

専門用語をやさしい言葉で説明

最終章 3/4。日常の言葉に置き換えます。

言葉	やさしい説明
JSON	名前と値を組にして情報を渡す文字の形式です。
API	プログラム同士が情報を頼んだり渡したりする窓口です。
アップロード / rsync	ファイルをサーバーへ送り、内容をそろえる作業と道具です。
設定 / 環境変数	場所ごとに変わるURLや保存先などの値です。
共有秘密	originとtamanegiだけが知る長い文字列です。
HMAC署名	共有秘密でデータへ付ける電子的な印です。
ハッシュ / SHA-256	ファイル内容から作る照合値です。
原子的置換	新しいファイルを完成後に一度で入れ替える方法です。
ログ	何時に何が起きたかを残す記録です。
バックアップ / 復元	予備の写しを作ることと、その写しから戻すことです。

専門用語をやさしい言葉で説明

最終章 4/4。日常の言葉に置き換えます。

言葉	やさしい説明
systemd / service / timer	Ubuntuで仕事を管理する仕組み、仕事本体、開始用の時計です。
release / current	版ごとの完成物と、今使う版を示す目印です。
シンボリックリンク	別の場所を指す近道のようなファイルです。
リバースプロキシ	利用者とoriginの間に立ち、認証後だけ通信を渡す入口です。
セッション / Cookie	ログイン後の利用者を一時的に覚える仕組みと小さな目印です。
CSRF	本人が気付かず別サイトから操作させられる攻撃です。
ソース	人が読み書きする開発用の元ファイルです。
TypeScript / TSX / React	画面の動きと部品を書く言葉と仕組みです。
Node.js / pnpm / Vite	作業用PCで部品を集め、確認し、変換する道具です。
ビルド	開発用ソースをサーバーで使う完成物へ作り替える作業です。

分からないとき
言葉を覚えることより、どの機械で何をしているかを確認することが大切です。不明な値や命令は推測で進めず、担当者へ確認してください。